格致 SciZhi

面向科研过程的 AI 智能体工作台

把检索、抽取、综合、审查、归档连成一条可追溯、可干预、可复现的流水线。



科研智能体 · 研究者可控的证据流水线

0.9.0 本地候选 · Apache-2.0 开源许可 · Windows 安装包已构建

答辩演示：终端版 npm start，全程离线可复现

项目概览

一句话

格致把科研中最耗时间的几步——找资料、读证据、写综述、做复核、留档——串成一条可以暂停、可以追问、可以复跑的工作流。

谁会用

· 研究生与课题组：把一次综述从“反复翻找”变成“按站点推进”
· 企业研发与情报团队：数据在本机，过程有权限，结果可交接

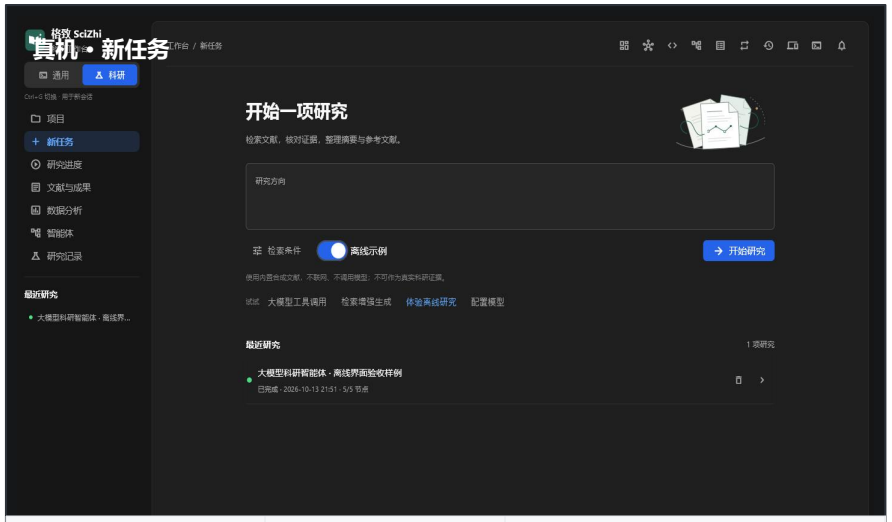
共同底座

Python 智能体引擎 + JSON Lines 桥协议；终端、桌面和手机远控共享同一份研究档案。

40+ h
传统综述事务性耗时（样例）

7 站
从立项到归档的科研流水线

1 份
可带走的研究包与证据账本



形态	入口	适合场景
终端工作台	npm start	快速推进
原生桌面	安装包 / 源码	长时间研究
手机远控	局域网 Web	看进度 / 审批

痛点：科研工作流断在四个地方

检索碎裂

OpenAlex、CrossRef、PDF、网页分别翻找，去重与引用追踪全靠手工。一次综述平均 40+ 小时，其中约七成是事务性劳动；检索式不留档，下一次从零开始。

证据失锚

结论散落在不同 PDF 页码与笔记里，引用与正文对不上。抽查一遍就露馅，投稿与答辩被追问「这条出自哪」时，只能回头人工核对。

过程不可复现

三个月后复跑同一课题：检索式没存、轮次没记、池子没有版本。答不出「上次为什么筛掉那篇」，人一毕业即断代，经验无法沉淀。

合规成本高

AI 生成内容没有标识，校/企合规红线过不去；模型密钥散落在各人的配置文件里；外部工具越权写入，出事查不到责任人。

缺口：缺少一个把五步串起来、且每一步都可追溯、可干预的科研工作台。

产品形态与技术架构

界面层

终端 TUI
桌面 Flet
手机远控 Web

研究者看到的是同一套命令与状态。

桥接层

src/bridge.py
JSON Lines
命令合约 + revision

界面不直接碰业务逻辑。

智能体引擎

Plan / Agent / Review
Checkpoint + HITL
检索、抽取、综述、审查

可无头运行、可单测。

数据与生态

插件 / Skill / GEZHI.md
MCP（stdio + HTTP）
git 研究档案 + 用量日志

产物带 manifest 与校验和。

一条边界原则

界面只发命令，引擎只产事件；每个事件都有权限、输入、输出和 revision。这样终端、桌面、手机可以换，证据链不丢。

隔离验证：GEZHI_DATA_DIR 指向临时目录，回归测试不触碰真实研究数据。

从点击到归档：一条可观测的事件链

```
UI → bridge.py → agent runtime → research package
          JSONL events · permission · checkpoint · manifest · sha256
```

科研流水线：七站，每一站都可回看

立项 研究方向与关键词，目标常驻上下文

假设 可证伪假设进假设台，证据加权排序

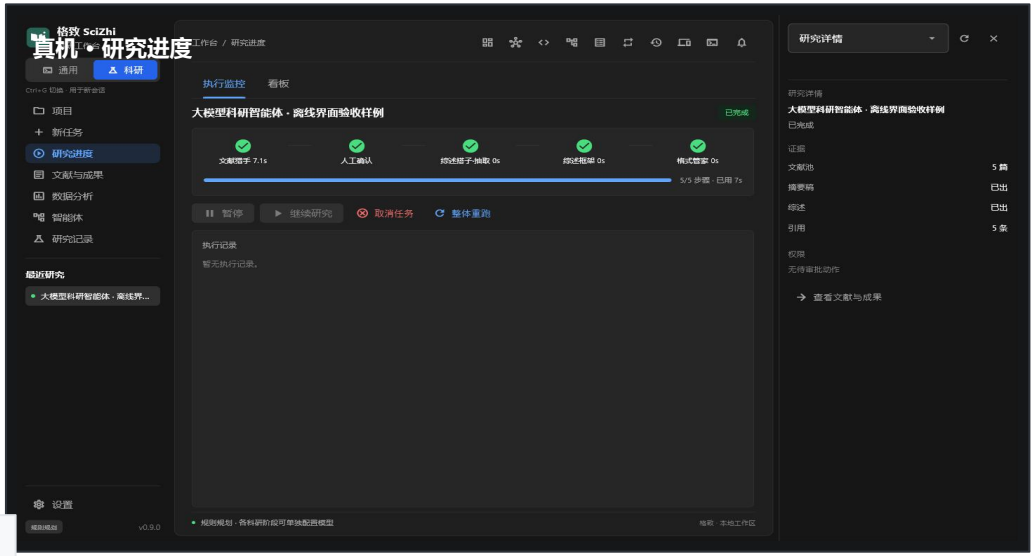
检索 双源并发召回 + 引用滚雪球递归

提取 结构化抽取，结论绑定文献出处

综合 综述框架 plan→act→reflect，缺证据自动补检

审查 AI 审稿人五维评分 + 后台证据复核

归档 研究包：manifest + 产物 + 数据 SHA256



任务完成度、五站状态和研究详情同屏可见

终端里看到的两条实时视图：阶段灯轨 + 任务监控

```
❯ /timeline
  ● 立项目标   目标：大模型智能体综述
  ● 证据池     15 条 · 支持 7 · 背景 6
  ○ 世界事实   用 /fact add <论断> 录入

❯ /status
  task=llm-agents-review
     节点 4/5 · 80%
     ⏸ 已挂起 · ▶ /resume
```

Checkpoint 不是“保存一下”：暂停、续跑、取消、重跑、删除都有明确语义；人工关口用编号菜单完成决策，并把选择写回事件流。

文献猎手：双源检索 + 三级漏斗

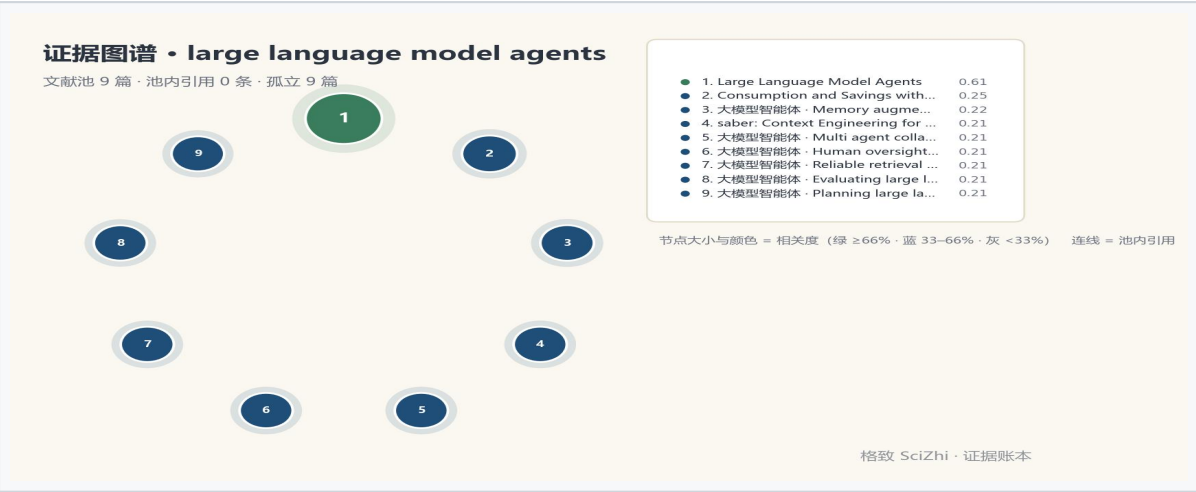
召回：两个源并行，一条链递归

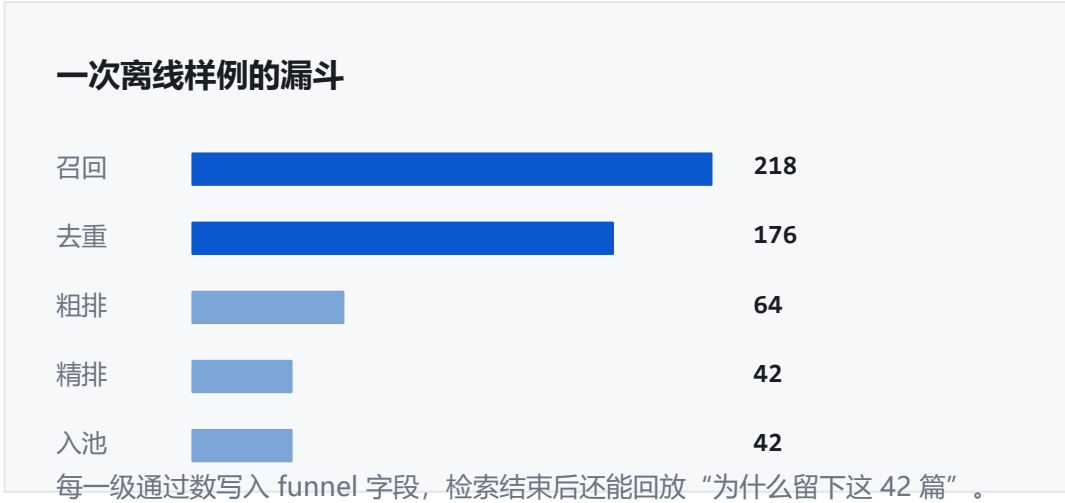
· OpenAlex 与 CrossRef 并发检索，DOI 去重，单源失败自动降级
· 引用滚雪球：正向被引 + 反向参考文献双侧扩展
· 收敛判断：新增低于阈值即停止加深，不做无限递归
· 去重键：DOI 优先，缺失时回退「标题 + 年份」模糊匹配
· 轮次级断点存档，中断后从 round-N 续跑

排序：两级粗排 + LLM 终审

· 默认 0.6×TF-IDF 余弦 + 0.4×饱和 BM25（纯 Python Okapi）
· 配置 Embedding 端点后升级为混合评分，失败自动降级
· 离线行为逐比特不变，评测脚本可对照两种模式
· LLM 语义终审保留否决权；否决后不足 5 篇则放弃否决，防空池
· 每级通过数写入产物 funnel 字段，可展示、可回放

结果视图 • 证据图谱





每一级通过数写入 funnel 字段，检索结束后还能回放“为什么留下这 42 篇”。

节点大小与颜色表达相关度；右侧列表保留出处

综述搭子、数据速览、格式管家、暂停键

综述搭子

先搭框架，再补证据；引用逐条回池核对，缺证据章节自动触发一次补检。

数据速览

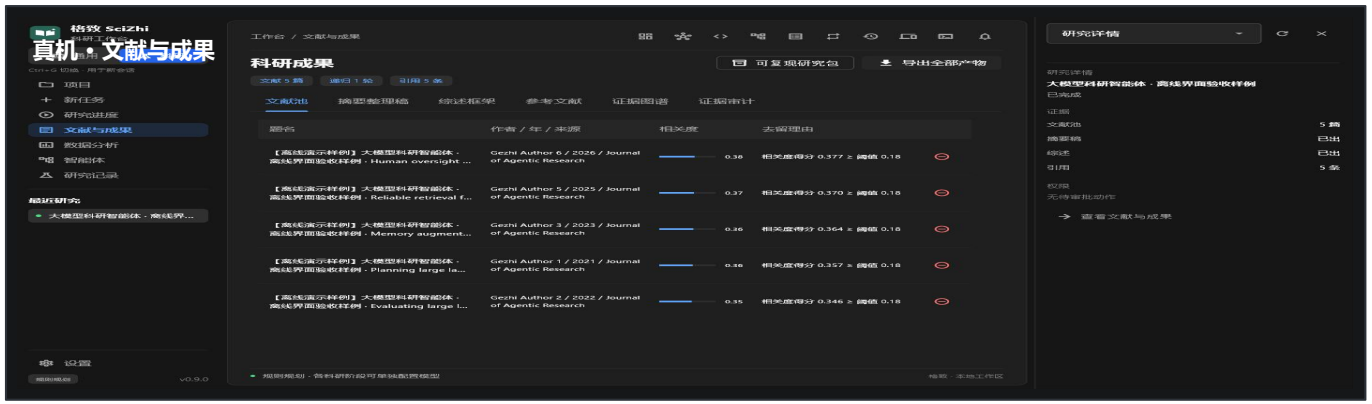
/stats 一次给出描述统计、IQR 异常值、相关性和图表建议，结果跟着研究包走。

格式管家

GB/T 7714—2015 与 APA 7 一键切换；元数据缺失就留空，不用模型“猜”。

暂停键

暂停、续跑、取消、重跑、删除五种语义写进事件流，人工确认后才继续。



文献、摘要、综述框架、证据图谱在同一研究包里汇总

一条可见的闭环

① 先给问题和边界
② 再由流水线召回与抽取
③ 最后把结论、引用和干预记录一起导出

AI 负责加速，研究者负责决定什么能写进最终版本。

自研模块：观测研究过程本身

不是把功能堆在一起，而是把研究对象连成一张图。

假设台

先问：这个想法值不值得做？

证据矩阵

再看：哪些文献支持、挑战或缺失？

同一份证据账本

实验树

把失败也留下，方便剪枝和复盘。

AI 审稿人

投稿前按结构、证据、可检验性自审。



每个节点都能回到文献、轮次与引用池

· 看板：池健康度、锚定率、孤儿文献、缺证据章节
· 新颖性检测：选题前先查差异点与风险，避免从已有答案开始
· 论文复现：白名单工具 + 研究包，让复跑有边界、有记录

智能体运行时：权限、审批与韧性

权限三档 × 三域

只读 / 工作区写入 / 完全访问，作用于引用 / 数据 / 代码；只读默认放行，写入与网络默认询问，策略可持久化（支持 research.* 通配符）。

子智能体与扇出

spawn 进程内派发，fanout 并行文献子任务：每路真实双源检索，子会话 JSONL 审计对父会话可见，单路失败降级。

审批卡的三个问题

目标是什么、影响面多大、是否可逆——审批时把三个答案摆在眼前；支持「批准并本会话记住」，重启后从事件记录恢复，已处理不可重放。

韧性四件套

上下文压缩（软阈值先摘要折叠）· 工具失败自愈（错误回填，2 轮上限）· 单步并行只读工具（≤4 并发保序）· 中断对账恢复（崩溃后按日志续跑）。

一条命令看清边界：规划不写盘，执行必过审批

```
❯ /plan 研究智能体          → 只读规划：拆解任务，不执行副作用
❯ /agent 研究智能体         → 执行模式：写入 / 网络工具触发审批卡
❯ /approve                  → 编号处理：1 放行 / 2 拒绝 / 3 看参数
```

一次审批的闭环



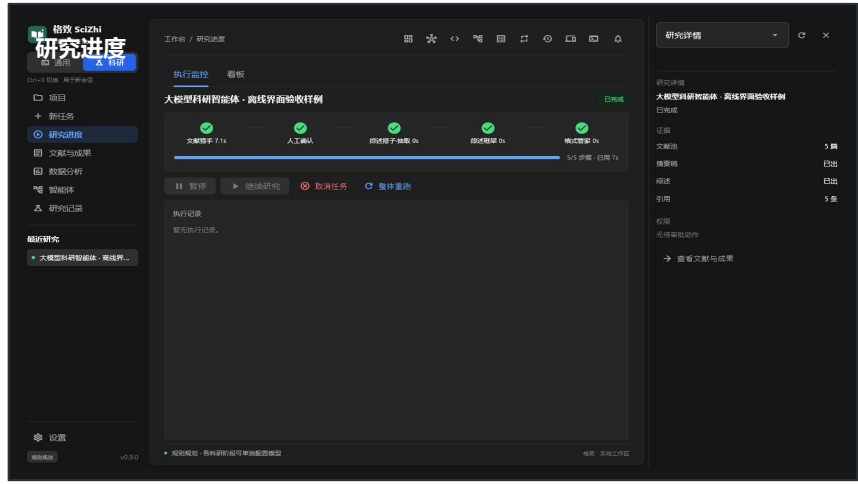
双端体验与界面设计



离线研究入口



计划、工具与人工确认



五站状态、详情和恢复入口

终端：93 条命令 + 三层补全，全键盘流

```
❯ /config          模型 / 主题 / 科研 / 界面（Enter 即切即存）
❯ @src/ranking.py  文件提及补全 → 自动进入上下文
❯ !pytest -q       shell 模式（项目根目录执行）
❯ /timeline        七站灯轨 + 空洞提示
❯ /read board      全屏阅读器：/ 搜索 · n 跳转 · q 返回
```

为什么认真做界面

· 科研会话常常持续数小时：信息密度优先，少滚动
· 面板吃掉终端高度：光标可见 > 上下文 > 装饰
· 终端、桌面、手机共享同一套状态和命令，不需要重新学习

安全、合规与可复现

密钥与权限

· 密钥仅显示「已配置」，原文不出本机；敏感命令不进输入历史、回显脱敏
· 工具权限三档 + 每智能体白名单；按会话文件锁，多进程审批互斥
· 研究数据可用 GEZHI_DATA_DIR 完全隔离；远程模式危险操作交回主机

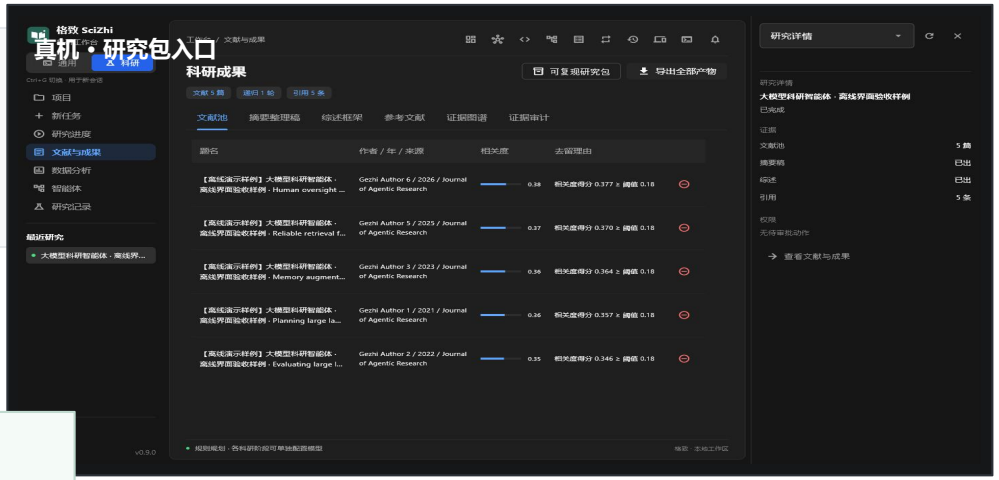
合规标识（依据《人工智能生成合成内容标识办法》）

· 摘要与框架正文标注：生成服务、生成方式、时间、任务标识、提示词版本
· 导出目录自动生成 manifest.md，逐文件标注 AI 参与度
· 确定性排版产物明确标注「非 AI 生成」

可复现：把「怎么做的」和「结果是什么」一起打包

研究包（一次导出即可带走全部过程）

`manifest.md`	产物清单 + AI 参与度 + 生成参数
`*.json / *.csv`	断点、漏斗、文献池、事件流
`data.sha256`	数据与产物校验和；研究档案库 `git` 逐轮提交（署名），过程有 `diff` 可回滚



安全性有证据，开源有连接。

持续更新仓库 • github.com/abzzyyxcpht/gezhi-scizhi • Apache-2.0 • README / CHANGELOG / CI

工程质量与验证基线

1691

项 Python 断言全绿（2 跳过，160 个测试文件）

82

个 TUI 测试文件；单链 62 项通过，含无头真帧断言

93

条 REPL 命令，每条有描述与可用性合约



真帧与人工关口进入回归口径

测试怎么写的

· 纯函数不变量优先：几何 / 预算 / 滚动 / 光标、漏斗口径、引用格式、共识计量都有单测
· 真帧集成：无头渲染抓帧断言（面板分页、窄屏 54×18、人工关口菜单、夹具失败清理）
· 隔离与并发：临时数据目录、按会话文件锁、僵尸进程清理回归；Ruff 与 UI 静态校验进 CI 口径

开箱可用

· /doctor 九项自检：数据目录、依赖、文献源连通、模型端点与清单、提示词、MCP、git、缓存、断点存档、向量检索
· 离线可完整演示（内置公开语义样例，逐比特可复现）；Windows 安装包含 SHA256 校验

优势总结

维度	Codex	Claude Code	OpenClaw（小龙虾）	格致 SciZhi
默认定位	编码交付	终端编码	多渠道通用助手	**科研过程工作台**
双源文献检索	需工作流	需工作流	可扩展	**内置：OpenAlex + CrossRef**
证据链追溯	需外接	需外接	需外接	**内置：账本 + 图谱 + manifest**
断点 / 人工关口	有审批模式	有权限控制	取决于配置	**Checkpoint + HITL**
科研模块	通用工具	通用工具	通用工具	**假设台 / 实验树 / 审稿人**
移动 / 远控	以代码环境为主	以终端为主	多渠道原生	**局域网 Web 远控**
本页口径	公开定位	公开定位	公开定位	**本项目实测流程**

科研专用链路

OpenAlex + CrossRef → 证据账本 / 图谱 → manifest

可控可审计

三档 × 三域权限 + Checkpoint / HITL + 事件流

可交接可复跑

研究包 + data.sha256 + git diff / 回滚 + 离线复现

注：比较的是产品默认定位与本项目演示口径，不构成基准性能排名。

进度与下一步

已完成（2026-08 → 2026-10）

- M1 检索骨架
- M2 综述闭环
- M3 双端体验
- M4 对标收尾

检索 / 去重 / 漏斗 / 断点 · 抽取 / 框架 / 防幻觉 / 补检 · 双端与手机远控 · 单引擎 / 权限三档 / 研究包

下一步（1.0 版）

插件生态
开放市场 + 沙箱 + 权限清单

团队协作
共享证据池 + 审批流 + 角色

跨平台验收
macOS / Linux · IME · 辅助功能

真实评测集
跨学科对照 + 供应商盲测

目标：把「可追溯科研」从个人工具做成课题组基础设施——每个结论都能回到文献、命令与轮次。

团队分工

姓名	角色	负责内容
（填写姓名）	组长 · 主责	系统架构与核心引擎：智能体运行时（规划 / 执行 / 审查）、编排器与断点 / 人工关口、双源检索与排序、双端联调；路演主讲与技术方案定稿
（填写姓名）	成员 · 文献与证据	文献检索与证据链：OpenAlex / CrossRef 接入、去重与漏斗指标、引用滚雪球与证据账本字段
（填写姓名）	成员 · 桌面端界面	桌面端界面与设置中心：Flet 页面、主题与字体、审批卡与任务进程面板
（填写姓名）	成员 · 文档与演示	文档与演示材料：README 与使用指南、路演 PPT、演示脚本与录屏、答辩问答准备
（填写姓名）	成员 · 测试与合规	测试与合规：pytest 用例维护、隔离验证脚本、回归清单与系统自检项核对；参考文献格式（GB/T 7714 / APA）校对、AI 内容标识文案、赛题材料整理与现场后勤

分工按模块边界划分：核心引擎与系统集成由组长主责，并统筹进度与联调；其余成员各自独立负责一个功能模块，可并行推进、互不阻塞。
协作方式：研究档案库 git 逐轮提交留痕；每周一次接口对齐（引擎 / 界面 / 文档 / 测试）；验收以测试基线与演示链路为准。